

Secure User Data Vault Using Capability Code Authentication and AES-256

Encryption

Module: Cloud Computing

Student: Agata Gabara

Date: July 2026

1. Introduction

Cloud computing applications require effective mechanisms to protect sensitive user information against unauthorized access. Traditional authentication methods based only on usernames and passwords are often insufficient to ensure adequate security. Therefore, additional authentication mechanisms and encryption techniques are commonly used to enhance data protection.

This project presents the development of a Secure User Data Vault, a web application implemented using the Flask framework. The application combines password hashing with bcrypt, AES-256 encryption through the Fernet library, and capability code authentication to provide secure storage and controlled access to sensitive user information.

The application allows users to register, securely log in using three authentication factors, and access encrypted information only after successful verification.

2. Project Objectives

The main objectives of this project are:

- Develop a secure cloud-based web application using Flask.
- Store user information securely in a SQLite database.
- Protect passwords using bcrypt hashing.
- Encrypt sensitive user information using AES-256 (Fernet).

- Implement capability code authentication.
- Demonstrate secure registration, login, and protected data access.

3. Technologies Used

Technology	Purpose
Python	Programming language
Flask	Web application framework
Flask-SQLAlchemy	Database ORM
SQLite	Database management
bcrypt	Password hashing
cryptography (Fernet)	AES-256 encryption
HTML	User interface
CSS	Application styling

4. System Design

The Secure User Data Vault consists of the following components:

- User Registration Module
- Login Authentication Module
- Capability Code Generator
- Password Hashing Service
- AES-256 Encryption Service
- SQLite Database
- Secure Dashboard

The application follows a secure workflow where user information is validated, encrypted, and stored safely in the database. Access to protected information is granted only after successful authentication.

5. System Implementation

5.1 User Registration

During registration, the application validates all user inputs before processing.

The following security mechanisms are implemented:

- Password hashing using bcrypt.
- AES-256 encryption of sensitive information.
- Generation of a unique capability code.
- SHA-256 hashing of the capability code.
- Secure storage in SQLite.

Figure 1. Secure User Registration Interface

Caption

The registration interface allows users to create a secure account by providing a username, email address, password, and sensitive information. User credentials are securely processed before being stored in the database.

Secure User Data Vault

SQL Injection Protection with AES-256 Encryption

Username

Email Address

Password

Sensitive Information

Register Securely

5.2 Capability Code Generation

After successful registration, the application generates a unique capability code.

The capability code:

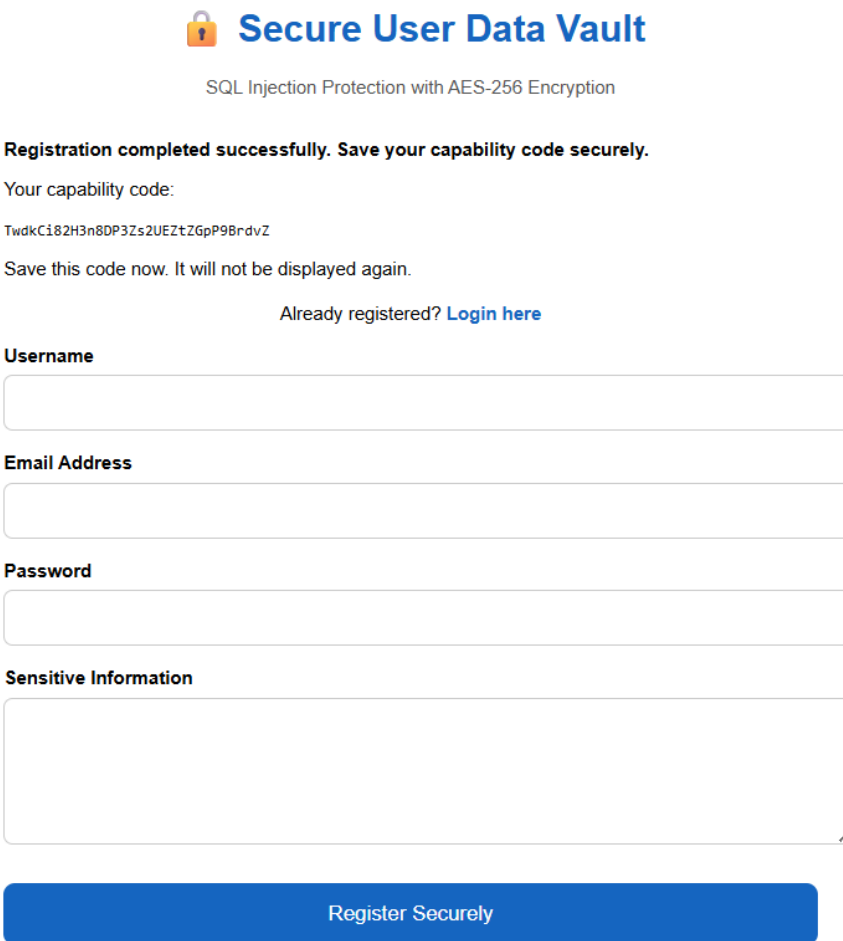
- is generated using a cryptographically secure random generator;
- is displayed only once;
- must be securely stored by the user;
- is required during every login.

Only the SHA-256 hash of the capability code is stored in the database.

Figure 2. Successful Registration with Generated Capability Code

Caption

After successful registration, the application generates a unique capability code. The code is displayed only once and must be securely saved because it is required for future authentication.



 **Secure User Data Vault**

SQL Injection Protection with AES-256 Encryption

Registration completed successfully. Save your capability code securely.

Your capability code:

TwdkCi82H3n8DP3Zs2UEZtZGpP9BrdvZ

Save this code now. It will not be displayed again.

Already registered? [Login here](#)

Username

Email Address

Password

Sensitive Information

Register Securely

5.3 Secure Login

The login process requires three authentication factors:

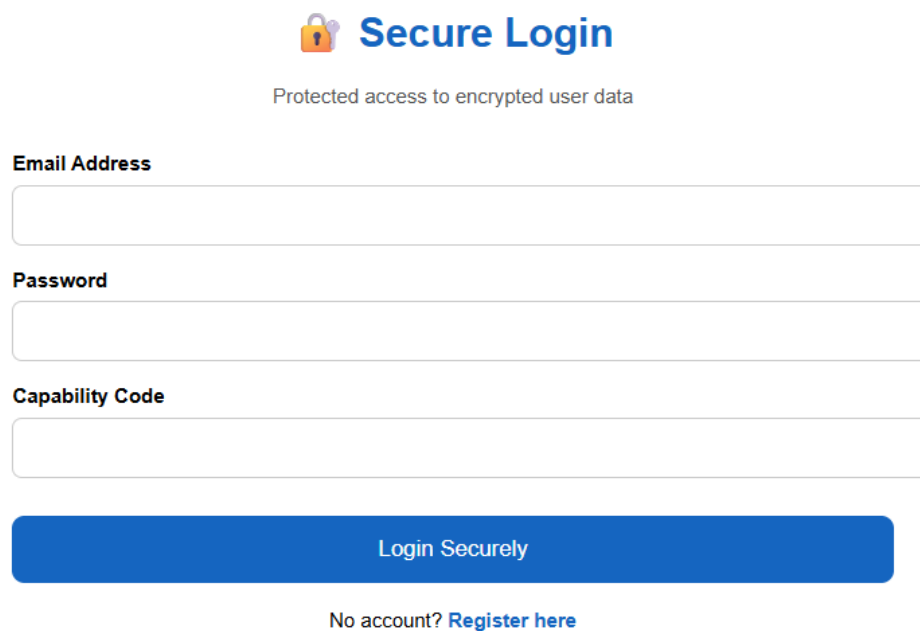
- Email address
- Password
- Capability code

The application verifies all three credentials before allowing access.

Figure 3. Secure Login Interface

Caption

The login interface requires users to enter their email address, password, and capability code. Access is granted only after successful verification of all authentication credentials.



The image shows a web form titled "Secure Login" with a lock icon. Below the title is the text "Protected access to encrypted user data". The form contains three input fields: "Email Address", "Password", and "Capability Code". Below these fields is a blue button labeled "Login Securely". At the bottom, there is a link that says "No account? Register here".

 **Secure Login**

Protected access to encrypted user data

Email Address

Password

Capability Code

Login Securely

No account? [Register here](#)

5.4 Capability Code Verification

The capability code is verified by comparing the SHA-256 hash of the provided code with the stored hash in the database.

The comparison is performed using secure hash verification to prevent timing attacks.

Figure 4. Successful Registration with Generated Capability Code

Caption

The application confirms successful registration by displaying the generated capability code. The capability code functions as an additional authentication factor that enhances account security.



Secure User Data Vault

SQL Injection Protection with AES-256 Encryption

Registration completed successfully. Save your capability code securely.

Your capability code:

fNIG_nm-dnGNRFuRTDwuu75y_rgT8uWY

Save this code now. It will not be displayed again.

Already registered? [Login here](#)

Username

Email Address

Password

Sensitive Information

Register Securely

5.5 Secure Dashboard

After successful authentication, the application decrypts the user's encrypted information using AES-256.

The dashboard displays:

- Username
- Email address

- User role
- Sensitive information

Only authenticated users can access this page.

Figure 5. Secure User Data Vault Application Workflow

Caption

The workflow illustrates the complete process of user registration, password hashing, AES-256 encryption, capability code generation, secure storage, user authentication, and protected data access.

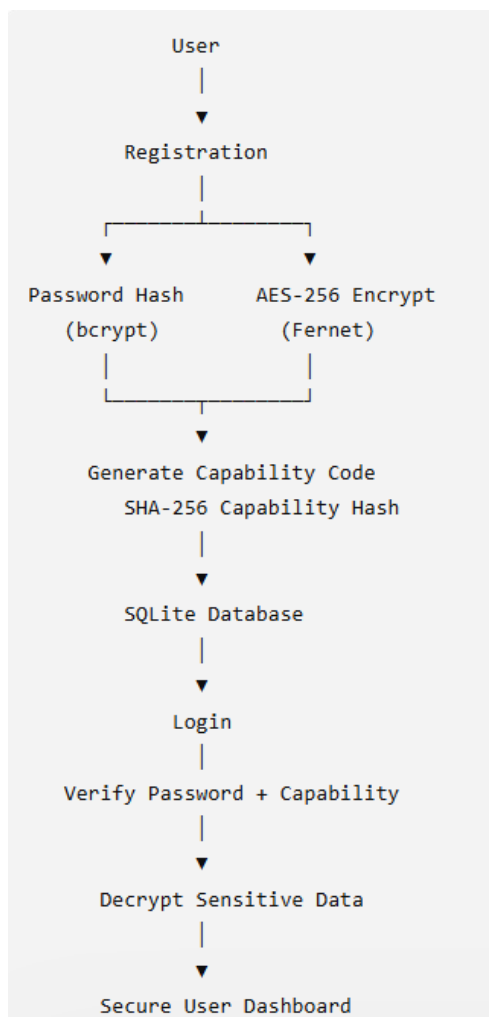
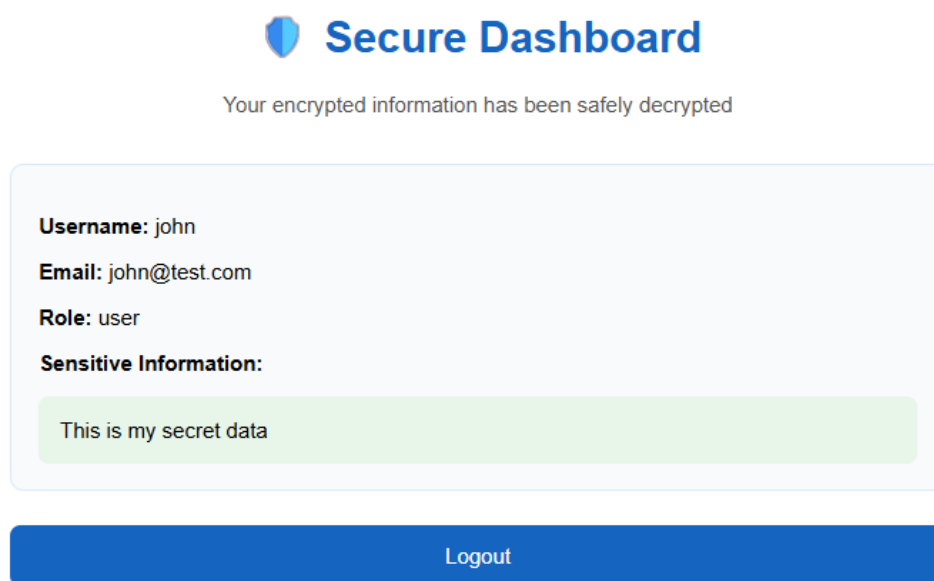


Figure 6. Secure Dashboard Displaying Decrypted User Information

Caption

After successful authentication, the application decrypts and displays the user's protected information, including the username, email address, user role, and sensitive information. This demonstrates secure storage, controlled access, and AES-256 encryption.



6. Security Features

The Secure User Data Vault implements several security mechanisms to protect user information.

These include:

- encrypt password hashing
- AES-256 encryption using Fernet
- SHA-256 hashing of capability codes
- Capability code authentication
- Session management
- Secure login verification

- Protected dashboard access
- Secure database storage

Together, these mechanisms provide confidentiality, integrity, and controlled access to sensitive information.

7. Results

The developed application successfully demonstrates a secure cloud-based authentication system.

The application allows users to:

- Register securely.
- Receive a unique capability code.
- Log in using three authentication factors.
- Store encrypted information securely.
- Access decrypted information only after successful authentication.

Testing confirmed that encrypted information cannot be accessed without the correct password and capability code.

8. Conclusion

This project successfully demonstrates the implementation of a secure cloud application using Flask, SQLite, bcrypt, AES-256 encryption, and capability code authentication.

The combination of password hashing, encryption, and multi-factor authentication significantly improves the security of stored user information. Sensitive data remains encrypted within the database and is decrypted only after successful verification of all authentication credentials.

Overall, the Secure User Data Vault meets the project objectives and demonstrates practical cloud security techniques that can be applied in modern web applications.

References

1. Flask Documentation. <https://flask.palletsprojects.com/>
2. Flask-SQLAlchemy Documentation. <https://flask-sqlalchemy.palletsprojects.com/>
3. Python Cryptography Documentation. <https://cryptography.io/>
4. bcrypt Documentation. <https://pypi.org/project/bcrypt/>
5. SQLite Documentation. <https://www.sqlite.org/>
6. OWASP Foundation. <https://owasp.org/>